

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: **Operating Systems**

COURSE CODE: **CSA04**

COURSE OUTCOME COVERED

***CO5:** Analyze the components and functionalities of Linux operating systems and virtualization environments to enhance their operational efficiency*

Assessment Tool Weightage: 35%

QUESTIONS: 5

Total Marks:25

Annexure A

CONSTRAINT BASED PROBLEM SOLVING

Code of Conduct:

*I **ANISH K/192521288** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



**Annexure A CONSTRAINT
BASED PROBLEM SOLVING**

S.No.	Question	Mark
1	A server with a total storage capacity of 50 GB needs to effectively manage space for rapidly growing system logs, user files, and application data. Analyze the scenario and design an optimized Linux file system layout, incorporating suitable partitioning techniques and storage management tools (such as log rotation), to ensure efficient utilization of disk space and maintain system stability. The solution must ensure that log files do not exceed 10 GB and that critical services continue to operate without interruption due to disk space exhaustion.	
2	A Linux system handling high CPU workloads suffers from frequent context switching delays. Given that hardware upgrades are not possible and real-time processing must be supported, evaluate kernel-level optimization techniques such as scheduler tuning and the use of kernel modules—to improve overall system performance.	
3	An administrator must process 1 million log entries per day using only shell scripting. Analyze the problem and design an efficient solution using optimized Linux commands that ensures the script completes within 2 minutes. Justify the choice of commands used for achieving high performance	
4	A Linux system runs multiple background services, where a faulty process is consuming 90% of CPU resources. Without restarting the system and while ensuring critical services remain unaffected, analyze the situation and propose a process management strategy using appropriate Linux tools to identify, monitor, and control the faulty process.	
5	A physical server must host 5 virtual machines with a total available RAM of 16 GB. Each VM requires a minimum of 2 GB, while the host operating system needs 4 GB. Analyze the scenario and propose an efficient resource allocation strategy using virtualization concepts to ensure all virtual machines operate smoothly under the given constraints.	
6	A Linux server has 16 GB RAM and runs several applications simultaneously. During peak hours, the system begins using swap heavily even though some applications appear idle. Analyze the memory-management behavior and recommend Linux commands and configuration techniques to identify the cause and optimize memory utilization.	
7	A Linux system contains thousands of files distributed across multiple directories. A backup process is taking several hours to complete. Analyze the directory structure and design an efficient command-line strategy using suitable Linux utilities to identify large files, duplicate files, and unnecessary temporary data.	

8	A Linux server has four CPU cores and supports multiple users running computationally intensive programs. Analyze how Linux scheduling manages these processes and propose a strategy using process priorities and CPU affinity techniques to improve responsiveness for critical applications.	
9	A system administrator discovers that the `/home` directory is consuming almost all available disk space while `/var` has significant unused space. Analyze the problem and design a suitable storage-management strategy that prevents user files from affecting system services.	
10	A Linux system becomes progressively slower after running continuously for several weeks. No hardware failure is reported. Analyze possible causes involving memory, processes, disk usage, and system resources, and propose a systematic troubleshooting procedure using appropriate Linux commands.	

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Optimized Linux File System Layout for a 50 GB Server

Problem Analysis

The server has only **50 GB** of storage, but it must handle:

- System logs
- User files
- Application data
- Rapidly growing log files

The major constraint is that **log files must not exceed 10 GB**, and critical services must continue running even when storage becomes full.

Proposed Partition Layout

Partition	Size	Purpose
/boot	1 GB	Boot files
/	10 GB	Operating system
/home	12 GB	User files
/var	10 GB	Logs and variable data
/opt	8 GB	Application data
/tmp	3 GB	Temporary files
Swap	4 GB	Virtual memory
Free/Reserved	2 GB	Emergency space
Total	50 GB	

Log Management

Since /var contains logs, **log rotation** should be configured using logrotate.

Example:

```
sudo nano /etc/logrotate.conf
```

A log can be rotated daily and compressed:

```
daily
```

```
rotate 7
```

```
compress
```

```
missingok
```

```
notifempty
```

This keeps old logs compressed and prevents unlimited growth.

Disk usage can be monitored using:

```
df -h
```

```
du -sh /var/log/*
```

Additional Protection

A separate /var partition prevents logs from consuming the entire root filesystem. Disk quotas can also be applied to user directories.

Conclusion

Separating /var, /home, and application storage prevents one type of data from affecting the others. **Log rotation, monitoring, quotas, and reserved space** ensure that logs remain below 10 GB and critical services continue operating.

2. Kernel-Level Optimization for High CPU Workloads

Problem Analysis

The system experiences frequent **context switching delays** during high CPU workloads. Hardware upgrades are not possible, and real-time processing must be supported. Therefore, kernel-level optimization is required.

Proposed Techniques

1. Scheduler Tuning

Linux scheduling parameters can be examined using:

```
sysctl -a | grep sched
```

The scheduler should prioritize processes that require real-time response.

2. Process Priority

`nice` and `renice` can control CPU scheduling priority:

```
nice -n -5 ./application
```

For an existing process:

```
renice -5 -p PID
```

A higher priority can be given to critical real-time processes.

3. CPU Affinity

CPU affinity prevents unnecessary migration of processes between CPU cores.

```
taskset -c 2 ./application
```

This runs the application on CPU core 2.

4. Real-Time Scheduling

Linux provides real-time policies such as:

```
chrt -f 50 ./application
```

This can give a critical process real-time scheduling priority.

5. Kernel Modules

Unnecessary kernel modules can be identified:

`lsmod`

Modules that are not required can be unloaded carefully:

`sudo modprobe -r module_name`

Conclusion

Scheduler tuning, CPU affinity, process priorities, and appropriate kernel modules can reduce unnecessary context switching and improve response time without upgrading hardware.

3. Processing 1 Million Log Entries Within 2 Minutes

Problem Analysis

The administrator must process **1 million log entries per day using only shell scripting**, with execution completed within **2 minutes**. Therefore, commands must be optimized and unnecessary processing must be avoided.

Efficient Commands

Instead of processing the file line-by-line using a shell loop, use optimized tools such as:

- `grep` – filtering
- `awk` – processing fields
- `sort` – sorting
- `uniq` – counting duplicates
- `cut` – extracting fields

Example:

```
awk '$9 >= 500 {count++} END {print "Errors:", count}' access.log
```

To count specific errors:

```
grep -c "ERROR" application.log
```

To extract and count IP addresses:

```
awk '{print $1}' access.log | sort | uniq -c | sort -nr
```

Optimization

Avoid:

```
while read line
```

```
do
```

```
...
```

```
done < logfile
```

for one million records because repeatedly invoking shell operations is slower.

Instead, use a single `awk`, `grep`, or combined pipeline.

Example Script

```
#!/bin/bash
```

```
LOG="application.log"
```

```
echo "Total Errors:"
```

```
grep -c "ERROR" "$LOG"
```

```
echo "Critical Errors:"
```

```
grep -c "CRITICAL" "$LOG"
```

```
echo "Top IP Addresses:"
```

```
awk '{print $1}' "$LOG" | sort | uniq -c | sort -nr | head
```

Conclusion

Using **compiled Linux utilities such as grep and awk instead of shell loops** significantly reduces processing overhead and makes processing one million entries within the required time more achievable.

4. Managing a Faulty Process Consuming 90% CPU

Problem Analysis

A faulty background process consumes **90% CPU**, but the server cannot be restarted and critical services must remain unaffected. The process must therefore be identified, monitored, and controlled safely.

Step 1: Identify the Process

`top`

or:

`ps aux --sort=-%cpu | head`

This identifies processes consuming the highest CPU.

Step 2: Monitor the Process

`pidstat -p PID 2`

This provides CPU usage information for the selected process.

Step 3: Reduce Priority

If the process is not critical:

`renice 10 -p PID`

This reduces its CPU scheduling priority.

Step 4: Stop the Process Gracefully

`kill PID`

If it does not terminate:

`kill -9 PID`

SIGKILL should be used only when necessary because it does not allow the process to clean up.

Step 5: Control CPU Usage

For services managed by systemd, resource limits can be applied using:

```
systemctl set-property service_name CPUQuota=20%
```

Conclusion

The safest approach is:

Identify → Monitor → Lower priority → Gracefully terminate/restrict → Force kill only if necessary.

This controls the faulty process without rebooting the system or unnecessarily affecting critical services.

5. Resource Allocation for 5 Virtual Machines

Problem Analysis

The physical server has **16 GB RAM**.

Requirements:

- 5 VMs
- Each VM requires minimum 2 GB
- Host requires 4 GB

Required memory:

$$5 \times 2 \text{ GB} = 10 \text{ GB}$$

Host requirement:

$$4 \text{ GB}$$

Total:

$$10 + 4 = 14 \text{ GB}$$

Available:

$$16 \text{ GB}$$

Therefore:

$$16 - 14 = 2 \text{ GB remaining}$$

The question specifies these resource constraints.

Proposed Allocation

System	RAM
Host OS	4 GB
VM1	2 GB
VM2	2 GB
VM3	2 GB
VM4	2 GB
VM5	2 GB
Total	14 GB
Remaining	2 GB

The remaining 2 GB can be retained as a buffer for host overhead and temporary workload increases.

Virtualization Techniques

Memory ballooning can dynamically reclaim unused memory from VMs.

Dynamic allocation can provide additional memory to VMs when necessary.

CPU resources should also be distributed according to workload rather than equally if some VMs are more important.

Conclusion

All five VMs can run because the minimum requirement is **14 GB**, which is less than the available **16 GB**. Keeping 2 GB as reserve improves system stability.

6. Heavy Swap Usage Despite Applications Being Idle

Problem Analysis

The server has **16 GB RAM**, but it uses swap heavily even when applications appear idle. This may occur because Linux can move inactive memory pages to swap to make RAM available for caching or other purposes.

Step 1: Check Memory

```
free -h
```

This shows RAM and swap utilization.

Step 2: Check Swap

```
swapon --show
```

Step 3: Check Memory-Consuming Processes

```
ps aux --sort=-%mem | head
```

or:

```
top
```

Step 4: Check Swappiness

```
cat /proc/sys/vm/swappiness
```

A lower value generally makes Linux less aggressive about swapping.

Temporarily change it:

```
sudo sysctl vm.swappiness=10
```

To make it persistent, add:

```
vm.swappiness=10
```

to:

```
/etc/sysctl.conf
```

Then:

```
sudo sysctl -p
```

Step 5: Check for Memory Leaks

Long-running processes should be monitored for continuously increasing memory consumption.

```
ps aux --sort=-%mem
```

Conclusion

The administrator should **check RAM usage, identify memory-heavy processes, inspect swap usage, examine swappiness, and investigate memory leaks**. Reducing unnecessary swapping can improve system responsiveness.

7. Optimizing a Slow Backup of Thousands of Files

Problem Analysis

The system contains thousands of files distributed across multiple directories, and backup takes several hours. Large files, duplicate files, and unnecessary temporary data should be identified before backup.

Find Large Files

```
find /home -type f -size +500M -ls
```

This identifies files larger than 500 MB.

Another approach:

```
du -ah /home | sort -rh | head -20
```

Find Temporary Files

```
find /home -type f -name "*.tmp"
```

Old temporary files can be reviewed and removed if they are no longer required.

Find Duplicate Files

fdupes can identify duplicate files:

```
fdupes -r /home
```

Efficient Backup

Instead of copying everything every time, use:

```
rsync -av --delete /home/ /backup/home/
```

rsync transfers only changed files, reducing backup time.

Compression

For archival purposes:

```
tar -czf backup.tar.gz /home
```

Conclusion

The best strategy is to **identify large files, remove unnecessary temporary data, identify duplicates, and use rsync for incremental backups**. This reduces both storage consumption and backup time.

8. CPU Scheduling Using Priority and Affinity

Problem Analysis

The Linux server has **4 CPU cores** and multiple users run computationally intensive programs. Critical applications must remain responsive. Therefore, process priority and CPU affinity should be used.

Process Priority

Linux uses a nice value from approximately **-20 to +19**.

A lower nice value means higher scheduling priority.

Start a critical process with:

```
nice -n -5 ./critical_app
```

Change an existing process:

```
renice -5 -p PID
```

CPU Affinity

Check the current affinity:

```
taskset -p PID
```

Run an application on core 0:

```
taskset -c 0 ./critical_app
```

For another application:

```
taskset -c 1,2 ./program
```

Proposed Allocation

CPU Core	Workload
Core 0	Critical application
Core 1	Critical/background application
Core 2	User workload
Core 3	User workload

This prevents computationally intensive background tasks from occupying every core.

Conclusion

Using **nice**, **renice**, and **taskset** allows administrators to prioritize critical processes and control which CPU cores they use, improving responsiveness.

9. /home Full While /var Has Free Space

Problem Analysis

The /home directory is consuming almost all available space while /var has significant unused space. User files should not be allowed to affect system services.

Step 1: Identify Large Users

```
du -sh /home/*
```

Find the largest directories:

```
du -ah /home | sort -rh | head
```

Step 2: Apply Disk Quotas

Filesystem quotas can limit how much storage individual users can consume.

For example, quota configuration can prevent one user from filling the entire filesystem.

Step 3: Separate /home

A better long-term design is to place /home on a separate partition or logical volume.

Example:

```
/
├── /var
├── /home
└── /opt
```

If /home becomes full, /var and the root filesystem remain available for system services.

Step 4: Clean Unnecessary Files

Identify old or unnecessary files:

```
find /home -type f -mtime +90
```

Files should be reviewed before deletion.

Step 5: Logical Volume Management

If LVM is used, storage can be resized more flexibly.

Conclusion

The best solution is to **separate /home, implement user quotas, monitor disk usage, and use LVM where possible**. This prevents user data from consuming storage required by system services.

10. Linux System Becoming Slower After Several Weeks

Problem Analysis

The system becomes progressively slower after running for several weeks, although there is no hardware failure. Possible causes include:

- Memory leaks
- Excessive swap usage
- Too many processes
- High CPU utilization
- Disk space exhaustion
- Large log files
- I/O bottlenecks
- Resource exhaustion

The question specifically requires analysis of memory, processes, disk usage, and system resources.

Step 1: Check CPU

`top`

or:

`uptime`

Check load average and CPU-consuming processes.

Step 2: Check Memory

`free -h`

Check which processes consume the most memory:

`ps aux --sort=-%mem | head`

Step 3: Check Swap

`swapon --show`

Heavy swap usage can indicate memory pressure.

Step 4: Check Disk Space

`df -h`

Check directories:

`du -sh /*`

Step 5: Check Disk I/O

iostat

High I/O utilization may indicate disk bottlenecks.

Step 6: Check Running Processes

ps aux

Look for unnecessary, zombie, or continuously growing processes.

Step 7: Check Logs

journalctl --disk-usage

Large logs can consume significant disk space.

Troubleshooting Procedure

Monitor → Identify bottleneck → Analyze process → Take corrective action → Monitor again

For example:

High CPU

↓

Identify process using top

↓

Check process behavior

↓

renice / restart service / terminate faulty process

↓

Monitor performance

Conclusion

A systematic approach using top, free, ps, df, du, swapon, iostat, and journalctl can identify whether the slowdown is caused by **CPU, memory, processes, disk space, or I/O**. Corrective action can then be taken without unnecessarily rebooting the server.